Threat Model and Secure Configuration

A threat model and prioritized hardening plan for BookNest, a small appointment-booking web app. Method: data-flow diagram, STRIDE, and a 5×5 likelihood × impact ranking.

1. Scope and assumptions

The app

BookNest lets customers book and pay for appointments at a small local business. Staff manage bookings in an admin panel.

Stack (assumed)

React single-page app behind a CDN, Node.js REST API, PostgreSQL, Stripe for payments, an email service for confirmations. Hosted on one cloud provider.

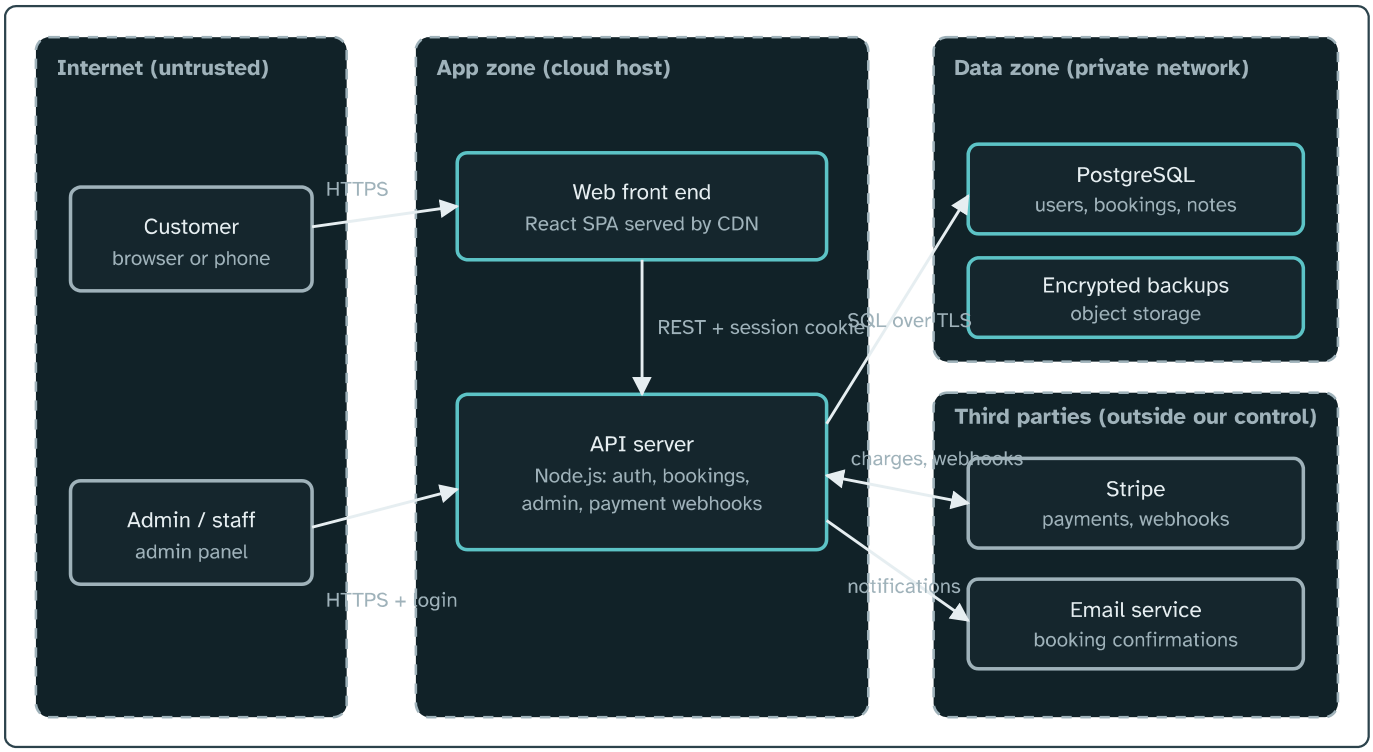
Assets worth protecting

- Customer names, emails, phone numbers
- Booking history and private notes
- Admin accounts and sessions
- API keys and database credentials
- Business reputation and uptime

Attackers considered: opportunistic bots, a curious or malicious customer, a stolen admin password.
Not considered: nation-state attackers and physical access to the data center.

2. Data-flow diagram and trust boundaries

Dashed boxes are trust boundaries. Every arrow that crosses one is a place where data must be checked.



Boundary crossings that matter most: Internet → App zone (all user input), App zone → Data zone (queries), Third parties → App zone (webhooks).

3. Threats (STRIDE)

S = Spoofing, T = Tampering, R = Repudiation, I = Information disclosure, D = Denial of service, E = Elevation of privilege. Likelihood (L) and Impact (I) are scored 1 to 5.

ID	STRIDE	Threat (abuse case)	Where
T1	S	Credential stuffing: bots try leaked email/password pairs on the login form and take over customer accounts.	Login en
T2	E, I	Broken access control (IDOR): a customer changes the ID in `/api/bookings/123` and reads or edits someone else's booking.	API
T3	T, I	SQL injection through search or filter parameters dumps or alters the	API → da

		database.	
T4	I	Secrets leak: a `.env` file is committed to Git or logged, exposing database and Stripe keys.	Repo, CI
T5	T, E	Stored XSS: a malicious script in the booking-notes field runs in the admin's browser and steals the session.	Notes fie panel
T6	R	No audit trail: an admin deletes or edits bookings and nobody can prove who did it or when.	Admin ac
T7	D	Bots flood login and booking endpoints, causing downtime and email-sending costs.	Public AP
T8	S, T	A forged payment webhook marks an unpaid booking as paid.	Webhook
T9	I	Verbose errors and missing security headers reveal stack traces, versions and internal paths.	API, fron
T10	T	Supply chain: a vulnerable or hijacked npm package gives an attacker code execution on the server.	Depende
T11	S, E	Admin account takeover through a reused password, with no second factor, exposes all customer data.	Admin pa
T12	T	CSRF: a malicious page makes a logged-in user's browser cancel or change a booking.	State-cha requests

4. Risk ranking

Score = Likelihood × Impact. Critical 15–25, High 10–14, Medium 6–9, Low 1–5. Sorted from highest risk.

Rank	ID	Threat	L	I	Score	Prior
1	T2	IDOR / broken access control	4	5	20	Crit
2	T1	Credential stuffing	4	4	16	Crit

3	T3	SQL injection	3	5	15	Crit
4	T4	Leaked secrets	3	5	15	Crit
5	T11	Admin takeover, no MFA	3	5	15	Crit
6	T5	Stored XSS	3	4	12	High
7	T7	Bot flooding (DoS)	4	3	12	High
8	T10	Vulnerable dependency	3	4	12	High
9	T6	No audit logging	3	3	9	Med
10	T8	Forged payment webhook	2	4	8	Med
11	T9	Verbose errors, missing headers	3	2	6	Med
12	T12	CSRF	2	3	6	Med

5. Prioritized hardening plan

Now (week 1): fix the Critical risks

- **T2:** On every endpoint, check that the logged-in user owns the record (`WHERE id = ? AND user_id = ?`). Use random UUIDs instead of sequential IDs.
- **T3:** Use parameterized queries or an ORM everywhere. Validate all input with a schema library.
- **T1, T7:** Rate-limit login and booking routes, add temporary lockout after repeated failures, and reject known-breached passwords.
- **T11:** Require MFA for every admin account and restrict the admin panel by IP or VPN if possible.
- **T4:** Move secrets to the host's secret manager, rotate any key that ever touched Git, and turn on secret scanning in CI.

Next (weeks 2–3): reduce High and Medium risks

Next (weeks 2–3): reduce High and Medium risks

- **T5:** Escape output by default, sanitize the notes field, and ship a strict Content-Security-Policy.
- **T10:** Commit the lockfile, enable Dependabot or `npm audit` in CI, and pin versions.
- **T8:** Verify Stripe's webhook signature on every call and ignore anything unsigned.
- **T6:** Log admin actions (who, what, when, IP) to an append-only store.

Later (month 2): baseline configuration

- **T9:** Return generic error messages, log details server-side, and set HSTS, X-Content-Type-Options and frame protection headers.
- **T12:** Set session cookies to `HttpOnly; Secure; SameSite=Lax` and add CSRF tokens to state-changing requests.